

# 面向高频 DApp 的 Hybrid TEE-Rollup 可恢复挑战协议与轻量 DA 成本评估

作者：杨帆

## 摘要

高频去中心化应用（Decentralized Application, DApp）如元宇宙状态同步、去中心化社交交互和链上 AI 推理服务，使区块链扩容问题呈现出典型的安全系统权衡：系统既需要较低交互成本和较低延迟，又不能在异常场景下放弃公开可验证性与挑战活性。围绕这一矛盾，本文研究 Hybrid TEE-Rollup 中一个范围较窄但关键的问题：当执行者仅向链上提交轻量承诺、完整负载数据位于数据可用性层，而挑战过程又因超时停滞时，协议如何恢复争议、重放证据并完成仲裁结算。本文构造并评估三个相互关联的机制接口：可恢复挑战机制、携带证据的紧凑提交以及数据可用性感知成本模型。基于 Python 原型、本地以太坊虚拟机燃料实测与 Sepolia 公开测试网部署记录，实验结果显示紧凑提交平均约为 406 字节；紧凑提交配合模块化数据可用性采样相比全链上调用数据在负载 8192、批大小 1000 时得到 96.33% 的配置化估算降本；可恢复挑战机制在合成超时场景下将挑战成功率从 0% 提升至 100%。上述结果仅用于说明机制接口、活性差异与成本趋势，不构成生产级安全或主网费用结论。

关键词：Hybrid TEE-Rollup；交互式挑战；数据可用性；高频 DApp；成本摊销；故障恢复

## 1 引言

高频去中心化应用的困难并不只是吞吐量不足。此类应用的单次交互价值通常不高，却要求持续状态更新、低延迟反馈、低链上负担，以及异常场景下的公开可验证仲裁。以链上 AI 推理服务为例，用户连续提交提示，执行者在可信执行环境中生成响应，并向链上提交紧凑提交；完整提示、响应与证明保存在数据可用性层。正常路径需要尽量

轻量，但异常路径必须具备足够的可验证性：当响应与数据可用性负载不一致，或挑战在中途超时，系统不能仅报告异常已检测，还需要保证争议能够继续推进到重放与结算。

现有路线分别缓解了上述矛盾的一部分，但难以同时满足低成本、低延迟、公开可验证和挑战活性四项要求。全链上执行保留了公开验证能力，却将高频负载、存储和执行成本直接推到链上；完全链下执行降低了延迟和成本，却缺少用户可依赖的公开仲裁路径；纯 TEE 执行可降低正常路径验证成本，但安全性和可用性仍受硬件和宿主影响。Hybrid TEE-Rollup 的优势在于将 TEE 快速路径与链上异常路径结合起来；相应地，其关键风险也在异常路径：如果缺少恢复机制和证据入口，系统可能在最需要仲裁时失去协议连续性。

超时后挑战流程卡死并非普通工程异常，而是协议活性与仲裁连续性问题。传统挑战机制可以关注错误是否能够被发现，但对于高频去中心化应用而言，检测只是争议处理的起点：未完成的挑战会占用状态、延迟结算，并使错误提交停留在不可结算的中间状态。因此，本文以故障发生、恢复、重放、仲裁结算为主线，将紧凑提交作为最小可验证证据入口，使正常路径保持轻量，异常路径仍可进入验证与重放。

实验部分采用合成负载、模拟 TEE、可验证数据可用性注册表、本地以太坊虚拟机燃料实测与 Sepolia 可部署性证据。实验设计与证据边界将在第 7 章和第 8 章集中说明。

## 1.1 本文贡献

本文的第一个贡献在于提出可恢复挑战机制。在传统挑战流程中，超时可能使争议停留在未解决状态，即使系统已经检测到异常，也无法自然推进到重放与结算。本文将恢复操作从普通的工程恢复提升为保持协议活性的状态转移机制：该操作不修改争议事实，在挑战事实不变的条件下将停滞的挑战会话恢复到可重放、可仲裁的路径上，从而保证了协议的仲裁连续性。详见第 4 章。

本文的第二个贡献在于设计携带证据的紧凑提交。该提交将状态根、输出哈希、证明哈希、数据可用性指针与数据可用性根绑定在同一上下文中，使链上对象同时具备重放准备性、证据可追溯性和面向验证的属性。详见第 5 章。

本文的第三个贡献在于构建数据可用性感知成本模型。该模型用于分析负载缩放、批处理摊销与数据可用性路径选择的权衡关系，帮助系统设计者理解紧凑提交方案在不同工作负载下的成本边界。详见第 6 章。

表 1 本文机制推进点与研究作用

| 机制推进      | 核心作用                  | 论文定位           |
|-----------|-----------------------|----------------|
| 可恢复挑战机制   | 在超时或中断后恢复争议流程         | 提升挑战活性         |
| 携带证据的紧凑提交 | 绑定状态/输出/证明/数据可用性根     | 形成最小证据入口       |
| 故障分类      | 区分证明、数据可用性、超时、重放和轨迹故障 | 明确异常责任边界       |
| DA 感知成本摊销 | 联合负载、批处理和数据可用性路径分析成本  | 支撑高频 DApp 成本论证 |

表 2 机制语义对比

表 2 的作用并非给出能力清单，而是界定本文机制与既有系统的关系：已有系统通常已经具备挑战机制或 TEE 快速路径，本文关注的增量在于 超时后的挑战活性、紧凑提交到 DA 证据的绑定关系，以及面向高频 工作负载的成本解释。

| 系统       | 挑战机制角色      | 恢复语义           | DA 证据绑定 | 成本模型作用 |
|----------|-------------|----------------|---------|--------|
| Optimism | 欺诈证明 / 争议博弈 | 主要依赖挑战窗口假设处理超时 | 非本文比较重点 | 本文未建模  |
| Arbitrum | 交互式争议定位     | 协议特定的争议推进机制    | 非本文比较重点 | 本文未建模  |

| 系统        | 挑战机制角色        | 恢复语义            | DA 证据绑定            | 成本模型作用         |
|-----------|---------------|-----------------|--------------------|----------------|
| OTR       | TEE 辅助的乐观验证   | 超时恢复讨论有限        | 部分负载/证明关联          | 本文未建模          |
| TEERollup | TEE 快速路径与挑战入口 | 主要作为系统机制讨论      | 架构层面讨论             | 未给出轻量数据可用性摊销模型 |
| 本文工作      | 可恢复挑战状态机      | 恢复操作作为保持活性的状态转移 | 紧凑提交到数据可用性字段级证明的绑定 | 负载/批处理/路径权衡分析  |

## 2 背景与威胁模型

### 2.1 汇总、可信执行环境与数据可用性

汇总的基本思想是将大量执行过程移至链下完成，并将状态承诺、证明或争议入口提交到链上。零知识汇总通过有效性证明保证提交状态的正确性，具有较强的验证属性，但证明生成、验证电路设计和复杂系统实现带来较高成本。乐观汇总假设链下执行结果默认正确，只有在挑战期内出现有效争议时才对结果进行仲裁。乐观汇总的优势是实现门槛相对较低、链上负担较轻；其弱点是争议窗口和交互期间的状态不确定性。

TEE 远程证明机制 为链下执行可信性提供补充接口。可信执行环境可以在隔离环境中执行程序，并对输入、输出、环境标识和 `nonce` 生成证明，使外部验证者能够确认某一输出来自指定执行环境。然而，TEE 更适合作为降低正常路径验证成本的组件，而不应替代链上挑战、数据可用性和故障追责机制。

乐观挑战是汇总系统中处理异常的重要机制。典型流程包括提交状态承诺、开启挑战、交互式缩小争议范围，并在必要时执行单步重放或链上仲裁。对于长执行轨迹，二分定位能够将争议定位复杂度从线性扫描降低到对数轮次；但 挑战本身也可能遭遇超时、应答缺失或证据不可用，因此其活性和恢复性需要被纳入协议设计。

数据可用性关注链下或链外数据是否能够被需要的验证者获取。模块化区块链和惰性账本等路线强调将数据可用性从执行验证中解耦，使共识层主要负责数据排序和可用性保证，而执行和验证由独立的汇总或客户端完成。当执行者将完整负载放在数据可用性层而非链上时，关键问题在于链上提交如何与数据可用性条目形成可验证的绑定。

## 2.2 问题范围与对抗假设

在上述背景下，本文进一步限定研究问题：**Hybrid TEE-Rollup** 如何在保持正常路径低成本的同时，保留公开、可恢复且有证据支撑的异常路径。成本维度要求链上提交不随完整负载线性增长；可验证性维度要求紧凑提交能够指向可检查的 **DA** 证据；活性维度要求 超时不会使仲裁流程永久停滞。

攻击面覆盖提交者、执行环境、数据可用性绑定和挑战流程。本文假设执行者或提交者可能提交错误响应、篡改数据可用性负载、提供无效证明，或在挑战流程中延迟响应；数据可用性层可能出现负载缺失、**Merkle** 证明损坏或根不一致；挑战流程本身也可能因交互超时而中断。本文不假设执行者在硬件 **TEE** 层面受到保护，也不假设数据可用性层具有无限可用性。

范围说明。本文的威胁模型不覆盖真实硬件侧信道、远程证明供应链故障、真实 **DA** 网络的经济安全、主网 最大可提取价值 以及拥堵导致的费用波动。该范围限定并不构成生产级声明，而是说明当前原型证据能够支撑的内容：协议状态连续性、证据绑定关系、配置化成本趋势，以及合约层面的对应关系。

## 3 系统概览

本节说明上述机制如何组成端到端系统。系统包含三个角色：执行层负责产生正常路径下的响应与 远程证明；**DA** 层负责保存重放 所需证据；验证层负责将故障转化为挑战、恢复、重放 与 结算 等状态转移。运行示例仍以高频 **AI** 推理 工作负载 为主：大量低价值请求不应迫使完整负载上链，但一旦发生争议，系统仍需保留公开仲裁所需的证据。

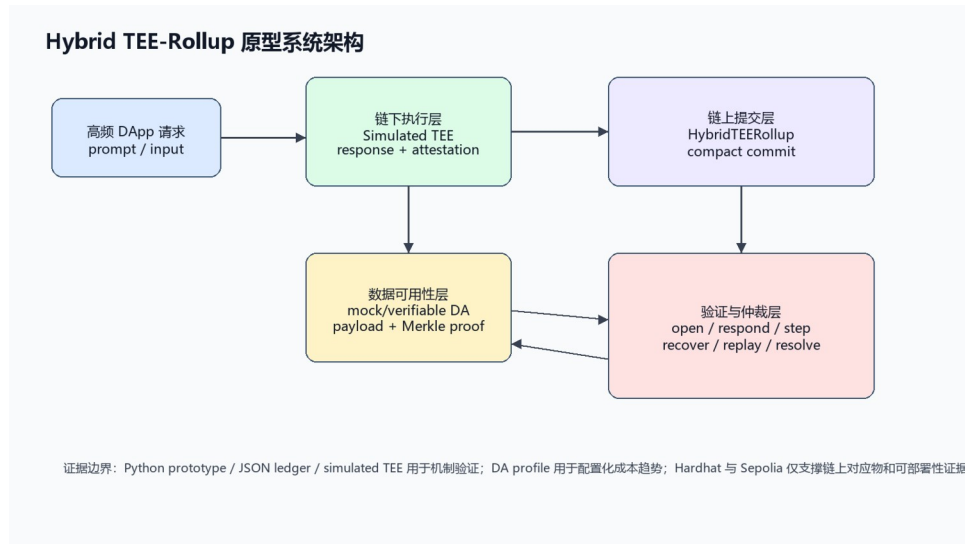


图 1 Hybrid TEE-Rollup 原型系统架构

如图 1 所示，系统由高频 DApp 请求入口、链下执行层、数据可用性层、链上提交层和验证仲裁层组成。正常路径中，执行层输出响应与 远程证明，数据层保存完整证据，链上提交层仅保存 紧凑提交。异常路径中，验证仲裁层通过 数据可用性指针 取回证据，并以 挑战、恢复、重放 和 结算 推进状态。这个划分使低成本正常路径与可仲裁异常路径保持同一证据上下文。

### 3.1 执行层与正常路径

执行层负责处理高频 DApp 请求。当前原型采用 确定性模拟模型 或可选模型后端生成响应，并使用 模拟可信执行环境 远程证明 对输入哈希、输出哈希、nonce 和执行环境标识进行绑定。远程证明的作用是为链下执行结果提供最小可信摘要，使后续验证层能够检查提交结果是否与声明的执行过程一致。

在本文原型中，模拟可信执行环境 的作用是建立“链下执行摘要—链上提交—异常挑战”的接口关系。真实 TEE 安全性不在本节展开，而在“讨论与局限性”中集中说明。

## 3.2 数据可用性接口

DA 接口是 紧凑提交 具有安全意义的关键位置。数据层保存完整负载 及其可验证结构，包括 提示、响应、远程证明、负载哈希、叶哈希、Merkle 根 和字段级 Merkle 证明；链上仅保存 紧凑提交。由此，字节压缩与证据保留被明确区分：正常路径避免完整负载上链，异常路径仍可恢复 重放 所需的负载上下文。

在运行示例中，如果执行者提交的响应被替换，验证者可以从 数据可用性条目 中取回 响应字段及其 Merkle 证明，并与链上 数据可用性 Merkle 根 检查一致性；如果负载 缺失，则异常应被归类为 数据可用性故障，而不应被误写为已完成 重放 的执行错误。

## 3.3 验证层与恢复触发

验证层负责处理异常路径。当前原型实现了 挑战打开、响应、步骤、恢复、重放和结算 等状态推进。其目标不是将所有执行都搬到链上，而是在出现争议时，通过证据加载、二分定位、单步重放和仲裁结算，将异常状态推进为可解释的结果。

验证层同时承担故障分类职责。本文将故障划分为证明故障（远程证明故障）、DA 故障、超时故障、重放故障和轨迹不一致。不同故障对应不同处理路径：数据不可用 表示数据层无法提供 负载；数据可用性证明无效 表示 负载 与链上 根 不一致；轨迹长度不匹配 表示执行轨迹与声明结构不一致；响应篡改 则可能同时导致 负载哈希、远程证明 和 状态根 不一致。

在 Solidity 映射中，原型包含 MockTEVerifier、MockDARegistry、MerkleVerifier 和 HybridTEERollup 等合约，用于表达链上状态推进和最小验证逻辑。本地 EVM 合约并非 Python 原型的逐行移植，而是抽取适合链上表达的提交、验证、挑战、恢复和结算路径。

## 4 可恢复挑战协议

### 4.1 协议模型与形式化定义

本节给出恢复机制成立所需的协议边界。恢复操作只有在恢复流程推进而不修改争议事实时才具有安全意义。下面的定义将会话状态与 挑战事实区分开，使 超时后的恢复可以被理解为活性层转移，而不是隐藏的正确性裁决。

#### 定义 1（挑战会话）

挑战会话定义为： $cs = (id, s, F, T)$ 。

其中， $id$  表示会话标识； $s$  表示当前状态； $F$  表示 挑战事实（挑战事实集合）； $T$  表示超时参数。

#### 定义 2（挑战状态机）

挑战状态机定义为： $M = (S, E, \delta)$ 。

其中， $S$  为状态集合； $E$  为事件集合； $\delta: S \times E \rightarrow S$  为状态转移函数。

#### 定义 3（恢复函数）

恢复函数定义为： $Recover(cs) = cs'$ 。

恢复仅改变活性相关状态，不改变 挑战事实，即满足： $F(cs) = F(Recover(cs))$ 。

### 协议性质

基于上述定义，恢复是活性层变换，而非正确性裁决。恢复函数允许会话从 TIMED\_OUT 等停滞状态回到可推进状态，但不重写紧凑提交、数据可用性根、负载哈希、轨迹或 不一致证据。该边界将恢复语义与 罚没语义区分开，并说明恢复操作不会改变 正确性：后续重放仍基于原始 挑战事实。

### 4.2 超时、恢复、重放与仲裁结算

可恢复挑战协议的目标，是将挑战从一次性错误检测流程扩展为异常场景下可恢复推进的协议状态机。其基本状态包括 PENDING、OPEN、RESPONDED、

NARROWING、RECOVERED、READY\_FOR\_REPLAY、REPLAYED、RESOLVED、SLASHED 和 FINALIZED。

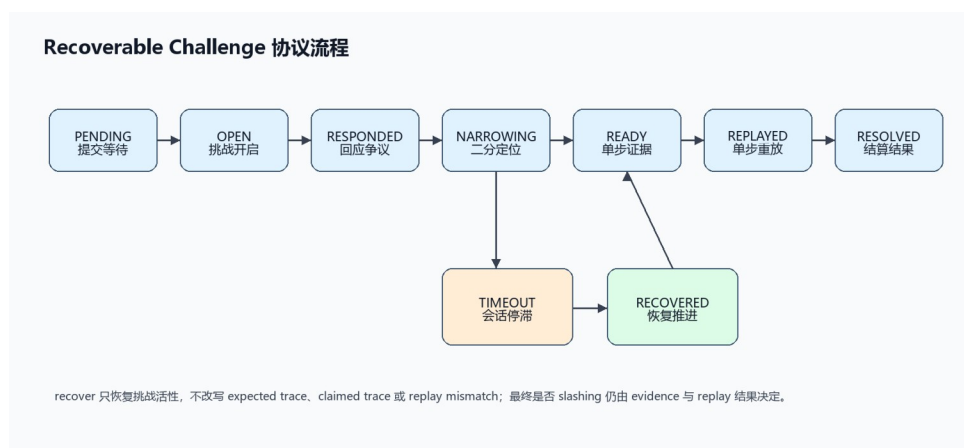


图 2 可恢复挑战协议流程

当交易处于待定状态时，挑战者可以调用打开操作进入已打开状态。系统加载紧凑提交、数据可用性状态和相关不一致证据。随后，被挑战方进入响应阶段，提供其对争议的回应。若争议涉及执行轨迹，协议进入二分步骤阶段，通过二分缩小定位争议区间。对于长度为  $n$  的轨迹，理想情况下定位轮次接近  $\log_2(n)$ ，从而避免对长执行轨迹进行线性扫描。

当争议区间被缩小到单步时，协议进入单步重放。重放阶段比较预期哈希与声明哈希，判断局部执行是否一致。如果重放失败，结算阶段可以将交易标记为罚没状态；如果重放通过，则挑战不成立或进入相应结算逻辑。

恢复是协议中的关键状态转移，因为它在不改变证据内容的前提下修复流程推进。在不可恢复的挑战中，超时可能使会话停留在“争议已知但无法重放和结算”的状态。可恢复挑战机制引入观察节点恢复路径，使超时会话重新回到可进行二分步骤、重放和结算的状态。这里需要区分活性与正确性：恢复操作恢复的是前者，而后续重放仍负责验证后者。

协议流程可分为三条路径。第一是正常路径：若挑战期内没有有效争议，交易最终进入最终确认。第二是可仲裁异常路径：若响应篡改或轨迹长度不匹配等执行相关故障

出现，协议通过打开、响应、二分步骤、重放与结算完成争议定位和结算。第三是超时恢复路径：若挑战过程因超时中断，观察节点可以调用恢复将会话推进到已恢复状态，并继续后续重放与结算。该划分有助于避免将异常检测、活性恢复和经济惩罚混写为同一性质。

### 4.3 正确性边界与协议性质

性质一，数据可用性绑定性质。紧凑提交中的 DA Merkle 根与 DA 条目中的字段级 Merkle 证明之间建立了密码学绑定关系，因此提示词、响应或证明字段被替换后，会导致负载哈希、叶哈希或 Merkle 证明与链上根不一致。该性质不依赖真实 DA 网络，只说明当前可验证 DA 结构能够检测字段篡改，并保证紧凑提交与完整负载之间在证据层面不可脱钩。

性质二，恢复机制的仲裁连续性。恢复操作的目标是恢复超时挑战会话的流程推进能力，使其继续进入二分缩小、重放和结算；它不修改预期执行轨迹、声称执行轨迹或重放不一致的事实。因此，恢复改变的是挑战流程的活性连续性，而非争议正确性本身。该性质将恢复语义与安全裁决语义明确分开，避免将“流程可以继续”与“裁决结果改变”这两个不同层次的性质混淆。

性质三，二分定位性质。对于长度为  $n$  的执行轨迹，二分挑战步骤每轮缩小争议区间，理想轮次接近  $\log_2(n)$ 。本文实验中轨迹步数为 4、8、16、32、64、128 时，平均二分轮次分别为 2、3、4、5、6、7，与理论对数趋势一致。该性质说明争议定位的开销增长是亚线性的，使得执行轨迹较长时挑战的交互成本仍可接受。

### 4.4 协议性质分析

前述讨论已将可恢复挑战机制的核心性质从“超时后可以重试”这一工程事实提升为协议层面的状态转移机制。本节进一步将这一认识系统化为挑战事实不变性、安全性与活性的分离、仲裁连续性和恢复操作正确性四项性质，并给出协议不变量表格，以明确恢复机制的协议语义边界。

### （一）挑战事实不变性

挑战会话在创建时便将一组不可修改的协议事实绑定在会话上下文中，这些事实构成争议的“内容面”，包括：

- 状态根（State Root）：约束输入与输出哈希的绑定关系；
- 数据可用性根（DA Merkle Root）：约束 DA 层中完整负载的字段级承诺；
- 负载哈希（Payload Hash）：标识 DA 层中完整的原始负载；
- 执行轨迹（Execution Trace）：挑战方与被挑战方各自提交的执行步骤序列；
- 重放证据：二分缩小后定位到的目标步骤与对应的预期哈希和声明哈希；
- 不一致证据：验证、响应和 DA 字段检查中观测到的字段级不一致记录。

记挑战会话 cs 的挑战事实集合为  $\text{Facts}(\text{cs})$ ，则恢复操作满足  $\text{Facts}(\text{cs\_before}) = \text{Facts}(\text{cs\_after})$ ，即恢复不向挑战会话注入新的状态根、负载哈希、执行轨迹或重放证据。

恢复操作恢复的是流程推进能力，而非修改争议内容本身。在不可恢复的挑战协议中，超时会使会话不可逆地停滞，已观测到的异常无法进入重放与仲裁。可恢复挑战机制将检测与处理之间的鸿沟打通，同时保证处理过程不脱离原始检测事实。

### （二）安全性与活性的分离

恢复机制设计中的一个关键考量是安全性与活性的边界划分。恢复操作解决的是活性问题——当挑战会话因交互超时而停滞时，恢复使其重新获得推进能力；但它不直接改变正确性判断——一个原本错误的执行结果不会因恢复而变为正确。

从安全性角度看，恢复操作受到以下约束：第一，恢复后的重放阶段仍对预期哈希与声明哈希进行一致性检查，两者不一致时挑战成功；第二，恢复不修改重放证据中的目标步骤，二分缩小定位到的争议点在恢复前后保持不变；第三，恢复不修改负载承诺，紧凑提交中的 DA Merkle 根和 DA 指针在提交时已锁定。

总结构成安全性的判断：恢复机制恢复的是协议活性，而非重新定义正确性。正确性的最终裁决由重放阶段完成，恢复只保证该裁决环节能够被执行。

### （三）仲裁连续性

仲裁连续性是本文提出的用于刻画挑战协议活性性质的概念：在挑战事实已经固定、重放证据可获得且数据可用性证明可检查的条件下，若争议已经被检测到，则协议应保证该争议不会因交互超时永久停滞，而能够继续推进至重放、结算或惩罚阶段。本文将满足这一性质的协议称为具备仲裁连续性。

传统基于超时的挑战协议可以完成异常检测——验证者观测到 DA 不一致、响应不匹配或证明无效后，能够将异常记录为争议。然而，当挑战方或被挑战方在交互中超时未响应时，争议可能永久停留在中间状态，交易既不能最终确认也不能被罚没。这就是检测性与完成性的差异：传统方案关注“能否发现异常”，仲裁连续性关注“已发现的异常能否被处理完毕”。对于高频去中心化应用，未完成的挑战会话会占用状态、延迟结算，使链上对象长期处于不确定状态，直接损害结算确定性。

可恢复挑战机制通过恢复函数在超时后恢复仲裁流程推进能力：观察节点调用恢复功能使超时会话回到可推进状态，从而继续二分缩小、重放和结算。实验部分（第 7.3 节）在合成超时场景下验证了恢复路径的效果——在当前原型设定下，恢复机制能够将超时停滞转化为可继续推进的仲裁流程。

**（四）恢复操作正确性与抗滥用边界**

恢复操作的协议安全性可以从两个角度考察：操作本身的正确性边界，以及恶意节点是否可能通过滥用恢复来破坏协议。两者可合并分析。

恢复操作允许修改的状态：超时计时器、会话推进状态标记和交互窗口。这些字段属于活性层，它们的修改使停滞会话重新获得推进能力。

恢复操作不允许修改的状态：重放证据（预期哈希、声明哈希和目标步骤）、执行轨迹承诺（二分缩小区间信息）、负载绑定（DA 指针与 DA Merkle 根）和数据可用性证明（Merkle 证明路径）。这些字段属于正确性层，恢复操作不能越过此边界。

由于恢复操作只能作用于已经链上承诺的挑战会话，恶意节点无法通过恢复重写不一致证据、注入新的执行轨迹或绕过重放阶段的正确性判断。反复恢复可能带来资源消耗问题，当前原型尚未实现恢复频率限制，留作后续工作。

**协议不变量与保持性质**

表 3 汇总了本文协议设计所依赖的核心不变量及其在恢复操作下的保持方式。这些不变量界定了恢复机制的安全边界：恢复操作被限制在活性状态空间内，不能跨越到正确性状态空间。

| 协议不变量   | 含义                                          | 恢复操作下的保持方式      |
|---------|---------------------------------------------|-----------------|
| 挑战事实完整性 | 状态根、数据可用性根、负载哈希、执行轨迹、重放证据和不一致证据在会话生命周期内不可修改 | 恢复操作不改变任何挑战事实字段 |

|         |                                                                     |                                   |
|---------|---------------------------------------------------------------------|-----------------------------------|
| 重放正确性   | 重放阶段对预期哈希与声明哈希的一致性检查是裁决正确性的唯一依据                                     | 恢复操作不修改预期哈希、声明哈希和目标步骤             |
| 仲裁连续性   | 在挑战事实已固定、证据可获得且数据可用性证明可检查的可仲裁争议中，协议应避免超时导致的永久停滞，并使争议继续推进至重放、结算或惩罚阶段 | 恢复操作是保持仲裁连续性的核心机制；将超时停滞会话恢复至可推进状态 |
| 数据可用性绑定 | 紧凑提交中的 DA Merkle 根约束 DA 条目的字段级内容                                    | 恢复操作不修改 DA 指针或 DA Merkle 根        |
| 执行轨迹一致性 | 二分缩小过程中积累的区间信息是重放定位的依据                                              | 恢复操作不重置或重写区间信息                    |

上述协议性质在实验部分（第 7.3 节）通过超时场景下不可恢复与可恢复两条路径的对比进行验证，重点考察超时后争议是否仍能推进至重放与结算。

## 5 携带证据的紧凑提交与数据可用性绑定

本节说明紧凑提交为什么不是单纯压缩结构。其研究价值在于携带异常路径所需的最小证据关联。该提交绑定状态根、输出哈希、证明哈希、数据可用性指针和数据可用性根：状态根约束输入、输出与环境摘要；输出哈希约束响应；证明哈希约束证明；数据可用性指针定位完整负载；数据可用性根则承诺字段级证明成员关系。

这种绑定关系使验证者能够从链上提交进入数据可用性证据结构。若响应被篡改，验证者检查响应叶与 Merkle 根；若证明与输出不一致，证明哈希和负载哈希形成交叉约束；若数据可用性证明损坏，系统可以将异常定位为证明无效，而不是混同为执行重放失败。由此，紧凑提交支持后续挑战和重放的证据加载，而不仅服务于链上字节压缩。

从对抗角度看，提交者可能替换负载字段、隐藏数据可用性条目、提交与证明不一致的输出，或在争议过程中延迟响应。紧凑提交本身并不能解决真实数据可用性网络可用性或真实 TEE 信任问题；其更窄的贡献在于明确链上最小承诺需要绑定哪些证据字段，才能使后续挑战具备重放准备性，而不仅是节省链上字节。

## 6 数据可用性感知成本模型

成本模型关注全链上与紧凑提交配合数据可用性在高频场景下的结构性差异。全链上路线将完整负载作为链上数据提交，其成本随负载大小增大近似线性增长。紧凑提交配合数据可用性将负载发布从链上调用数据转移到数据可用性配置，链上只保留紧凑提交，从而在高频大批量场景中控制成本增长。

模型比较四类数据可用性路径：全链上调用数据、紧凑外部数据可用性、紧凑类 EIP-4844 方案和紧凑模块化数据可用性采样。该比较属于配置化估算，应被理解为研究原型的成本建模工具，而非对真实主网定价的模拟。

模型表达为  $C_{total} = C_{commit} + C_{DA} + p_{challenge} \times C_{challenge}$ 。其中  $C_{commit}$  表示链上紧凑提交或完整负载提交成本， $C_{DA}$  表示不同数据可用性路径下的数据成本， $C_{challenge}$  表示发生争议时的挑战、恢复、重放和结算成本， $p_{challenge}$  表示争议发生概率。批处理后单笔摊销成本记为  $C_{amortized} = C_{total} / batch\_size$ 。

从系统角度看，批大小越大，固定提交成本越容易被摊薄；负载越大，全链上调用数据的线性成本越突出；数据可用性路径越接近低边际数据成本，紧凑提交配合数据可用性的收益越明显。因此，该模型的核心启示不在于某项数字，而在于批处理规模、负载大小和数据可用性路径选择三者之间存在结构性权衡。

## 7 实验评估

实验评估不仅验证原型能否运行，还围绕结果含义组织对比分析。需预先说明：当前实验使用模拟 TEE、合成负载、可验证数据可用性注册表和本地以太坊虚拟机基线，成本模型为配置化估算而非真实主网定价。这些边界在第 8 章集中讨论，本章聚焦于机制接口验证和趋势对比。RQ1 关注紧凑提交是否在保留证据关联的同时隔离负载增长；RQ2 关注批处理摊销与数据可用性路径选择的影响；RQ3 关注恢复机制是否改善

超时场景下的挑战活性；RQ4 关注不同故障类型能否被检测、恢复或惩罚；RQ5 则关注 Solidity 合约、本地实测与 Sepolia 部署能否支撑链上对应物。

$$C_{\text{total}} = C_{\text{commit}} + C_{\text{DA}} + p_{\text{challenge}} \times C_{\text{challenge}} \quad (1)$$

$$C_{\text{amortized}} = C_{\text{total}} / \text{batch\_size} \quad (2)$$

其中， $C_{\text{commit}}$  表示配置中的链上提交成本， $C_{\text{DA}}$  表示配置化数据发布成本， $C_{\text{challenge}}$  表示挑战、恢复、重放与结算的异常路径成本， $p_{\text{challenge}}$  表示争议发生概率。该模型用于不同数据可用性配置下的摊销比较，而非真实费用预测。

## 7.1 RQ1：紧凑提交能否降低链上提交规模

RQ1 隔离考察提交结构。Python 原型在模拟 TEE 下生成合成账本条目，比较完整负载与紧凑提交的规范 JSON 字节数。度量指标包括完整字节数、紧凑字节数和字节降幅比例。完整字节数表示完整提示、响应和远程证明都作为负载的一部分时的总字节数；紧凑字节数仅计算提交结构字节数。

对比结果显示了清晰的隔离关系：紧凑提交基本保持在 406 字节左右，而完整负载字节数随负载大小增大而增长。当负载大小为 8192 时，提示长度为 64 和 256 的完整负载字节数分别约为 50068.84 和 50262.84，而紧凑提交仍为 406.00 字节。该结果表明紧凑提交将负载规模扩展与链上提交规模分离，同时不丢失重放准备能力。

图 3 完整负载与紧凑提交的字节规模对比

## 7.2 RQ2：批大小如何影响 DA 感知摊销成本

RQ2 关注提交策略在何种条件下会改变高频合成负载的系统可行性。实验比较全链上调用数据、紧凑外部数据可用性、紧凑类 EIP-4844 方案和紧凑模块化数据可用性采样四类路线。主要指标为摊销燃料和降幅比例。

趋势表明，当负载发布成为主要开销时，批处理摊销的作用会更加明显。以负载 8192 为例，全链上调用数据的估算燃料从批大小 1 时的 801101.44 下降到批大小 1000 时的 801.21；而紧凑模块化数据可用性采样从 29026.98 降至 29.03。96.33% 是给定成本模型与数据可用性配置参数下的趋势结果，说明紧凑模块化数据可用性采样在大负载与大批处理条件下具有结构性优势。

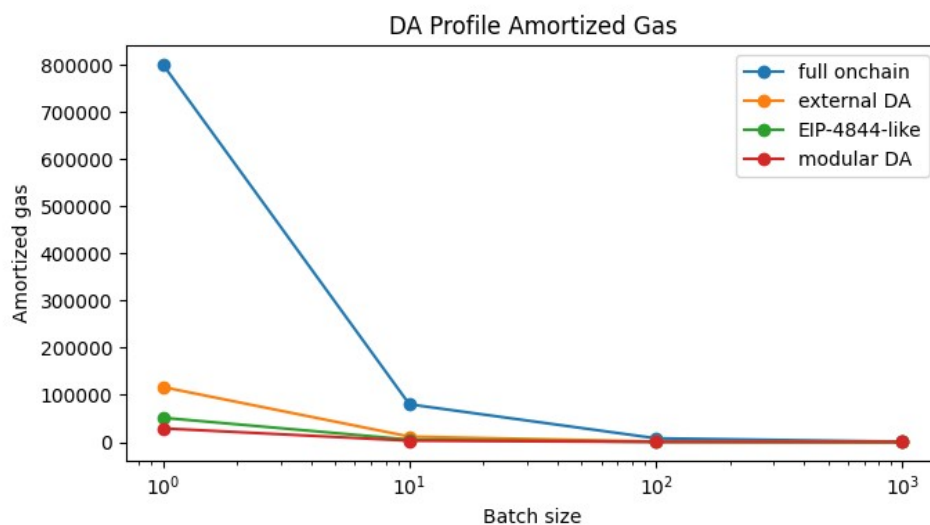


图 4 不同数据可用性配置下的摊销燃料趋势

## 7.3 RQ3：恢复机制能否改善 超时 场景下的挑战活性

RQ3 检验本文的核心活性主张。该实验对应第 4.4 节定义的仲裁连续性。Python 原型在不可恢复路径与可恢复路径两条路径中使用相同的合成挑战状态机，轨迹步数覆盖 4 至 128，挑战超时设为 2 秒。关键问题是：超时被检测后，会话是否仍能继续进入重放、结算与罚没。

二分定位行为与理论对数趋势一致。更关键的是不可恢复与可恢复路径的对比：在超时不可恢复场景中，检测率达到 100%，但挑战成功率和罚没比例均为 0%；在超时可恢复场景中，在当前原型合成负载下均可达到 100%。该对比说明，超时检测本身并不等价于协议推进。

### 7.3.1 恢复开销分析

利用表 5 中已有本地以太坊虚拟机数据，挑战步骤的平均燃料用量为 93807，挑战恢复为 156891，恢复相对单轮挑战步骤的开销比约为 1.67。该成本发生在异常路径，对比对象应为未解决仲裁与有界恢复转移之间的差异，而非正常路径吞吐与恢复开销。

该比例仅表示本地 EVM 中两条合约路径的相对开销。

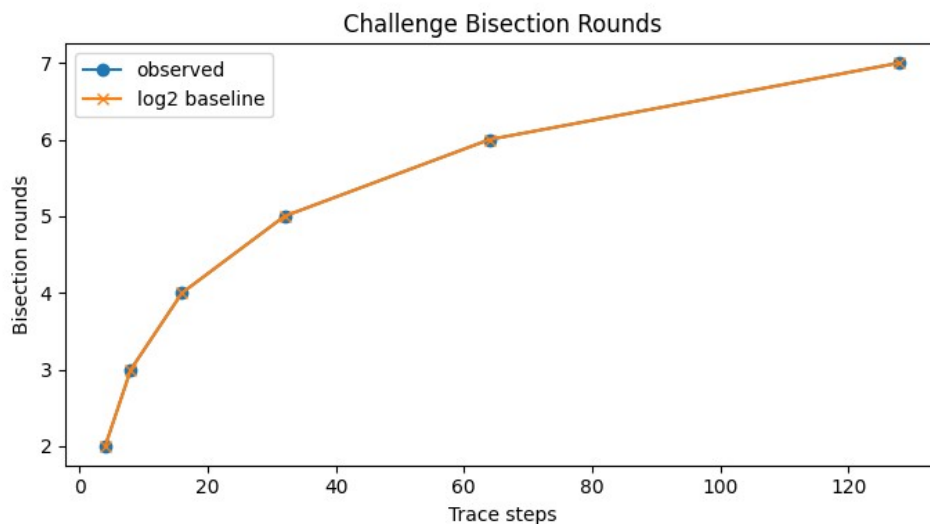


图 5 二分挑战轮次与 轨迹步数 的关系

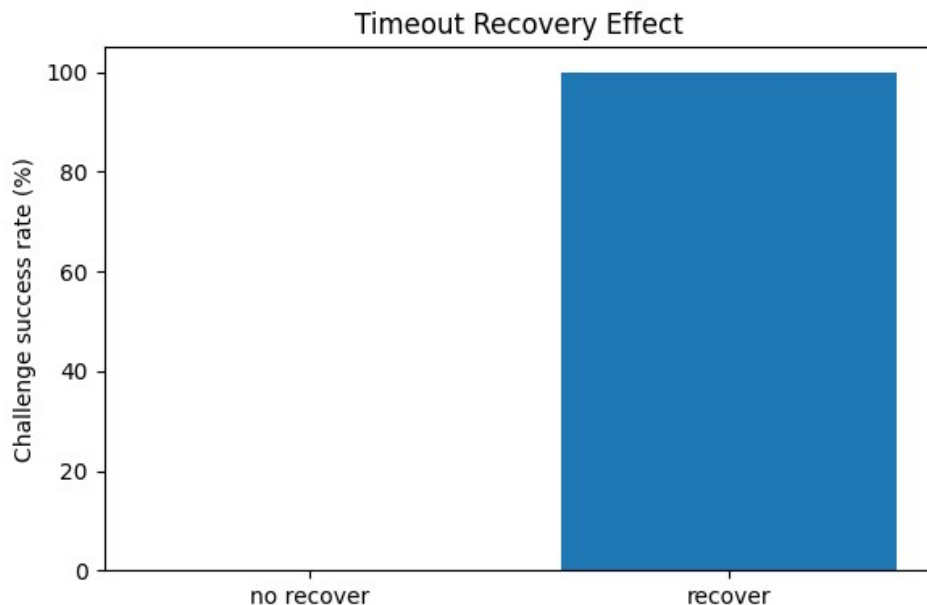


图 6 超时恢复对挑战完成率的影响

#### 7.4 RQ4 : 不同故障场景能否被检测、恢复或惩罚

RQ4 采用面向故障分类的安全评估方式。实验注入八类合成场景，区分其在当前原型中是否可检测、可恢复、可重放或可惩罚。指标为检测率、挑战成功率和罚没比例。

正常路径无故障，检测率为 0%，直接进入最终确认。响应篡改和轨迹长度不匹配在当前原型中可检测、可恢复、可重放和可惩罚，形成完整的挑战到罚没闭环。远程证明无效、数据不可用和数据可用性证明无效在当前原型中体现为检测和拒绝能力，尚未形成完整的挑战到罚没路径。超时不可恢复场景体现为检测但无法完成仲裁；超时可恢复场景则形成完整闭环。

图 7 失败场景检测与仲裁结果

表 4 失败场景分类与当前证据边界

| 类别        | 场景                       | 当前结果                    | 证据边界              |
|-----------|--------------------------|-------------------------|-------------------|
| 正常路径      | 正常路径                     | 不触发检测或罚没                | 说明正常路径不误报         |
| 可检测但未统一惩罚 | 远程证明无效, 数据不可用, 数据可用性证明无效 | 检测率 100%，挑战成功率和罚没比例为 0% | 说明可检测或拒绝，不说明已统一惩罚 |
| 挑战 + 惩罚闭环 | 响应篡改, 轨迹长度不匹配            | 检测率、挑战成功率和罚没比例均为 100%   | 说明典型执行异常可进入闭环     |
| 超时 活性对比   | 超时不可恢复 vs 超时可恢复          | 不可恢复路径 为 0%，恢复后为 100%   | 说明恢复改善仲裁活性        |

7.5 RQ5 : Solidity、本地 EVM 与 Sepolia 证据能否支撑链上对应物

RQ5 评估链上合约对应物。当前项目已将核心路径映射为 Solidity 合约，并在 Hardhat 本地测试链上得到燃料用量基线，见表 5。同一套合约已部署到 Sepolia 公开测试网，见表 6。

表 5 本地 EVM 关键路径平均 gasUsed

| 操作           | 平均 gasUsed | 解释                              |
|--------------|------------|---------------------------------|
| registerDA   | 113526.00  | MockDARegistry 记录 负载哈希 与 数据可用性根 |
| submitRollup | 299033.67  | 紧凑提交 提交路径                       |

| 操作                | 平均 gasUsed | 解释        |
|-------------------|------------|-----------|
| challengeOpen     | 283769.00  | 打开争议并加载证据 |
| challengeRespond  | 65886.00   | 被挑战方响应    |
| challengeStep     | 93807.00   | 二分定位的单轮推进 |
| challengeRecover  | 156891.00  | 恢复超时会话    |
| challengeReplay   | 135632.00  | 单步重放仲裁    |
| challengeResolve  | 113402.00  | 结算争议结果    |
| challengeFinalize | 33477.00   | 挑战期后最终确认  |

表 6 Sepolia 部署信息

| 项目                 | 数值                                         |
|--------------------|--------------------------------------------|
| Network / Chain ID | Sepolia / 11155111                         |
| 部署区块               | 10825904                                   |
| MockTEEVerifier    | 0xC42CB0Cf0D112Bd59E0f212F2DB2002ca50a8b6A |
| MockDARegistry     | 0xc42C64a7De05bf3ad4f1c75CbD8E0506F7De99Ff |
| HybridTEERollup    | 0xB9B72f10bB8aBC1ed090f1B0443Fe57189c497Fd |

实验的可复现性通过以下证据类型支撑：Python 原型脚本可生成完整实验输出；成本实验和挑战实验均使用确定性合成模型，不依赖外部推理服务；Hardhat 测试提供合约层面的可重复验证。所有实验输出文件（成本网格、挑战网格、故障场景、汇总文件）由同一实验脚本生成。

7.6 可复现性与证据类型

Python 原型位于 project\_code/tee\_rollup\_demo ，正式实验脚本为 project\_code/scripts/run\_paper\_experiments.py ，输出 cost\_grid.csv、challenge\_grid.csv、

failure\_scenarios.csv、summary.json 和 figures 目录。本地 EVM gas 脚本为 project\_code/evm/scripts/measure\_gas.js，输出 paper\_outputs/evm\_gas/measured\_gas\_report.md 与 measured\_gas\_summary.json。Sepolia 部署记录位于 project\_code/evm/deployments/sepolia.json。本次版本还保留 comparison\_figures 目录和 comparison\_experiments.md，用于记录 baseline comparison、fault-oriented comparison 与 数据可用性路径 tradeoff 的派生图表和说明。

复现实验可使用以下命令：python -m unittest discover -s tests -v；python scripts/run\_paper\_experiments.py --clean --samples 100 --payload-sizes 128,512,2048,8192 --batch-sizes 1,10,100,1000 --trace-steps 4,8,16,32,64,128；在 project\_code/evm 目录下运行 npm.cmd test 和 npm.cmd run measure:gas。上述命令分别对应机制验证、配置化成本趋势和本地 EVM 状态推进基线，不能相互替代。

7.7 对比实验小结

上述 RQ1 至 RQ5 已经包含了部分对比分析。本节将这些对比集中呈现，避免在不同 RQ 之间重复解释。以下仅列出关键对比维度与结论，详细数据参见对应 RQ 节和表格。

表 7 对比实验小结

| 对比维度   | 对应研究问题 | 基线        | 本文路径   | 主要观察                       |
|--------|--------|-----------|--------|----------------------------|
| 恢复活性对比 | RQ3    | 不可恢复路径    | 可恢复路径  | 超时检测不等于协议推进，恢复路径可继续进入重放与结算 |
| 提交规模对比 | RQ1    | 完整负载      | 紧凑提交   | 紧凑提交隔离负载增长并保留异常路径所需证据入口    |
| 故障分类对比 | RQ4    | 单一成功/失败判断 | 故障分类处理 | 不同故障对应检测、拒绝、重放或罚没等         |

|           |     |         |                |                           |
|-----------|-----|---------|----------------|---------------------------|
|           |     |         |                | 不同协议能力                    |
| 数据可用性路径对比 | RQ2 | 全链上调用数据 | 紧凑提交 + 数据可用性路径 | 成本边界受负载大小、批大小和数据可用性依赖共同影响 |

上述对比说明，本文机制的主要作用不是单一降低成本或提高检测率，而是将轻量提交、证据绑定与可恢复仲裁路径组合起来，使高频场景下的异常处理具有可推进性。

## 8 讨论与局限性

本文结论应理解为机制层研究原型结论。下面从执行环境、数据可用性、成本估算、链上实现和故障处理五个方面集中说明边界，避免在正文中反复声明。

### 8.1 执行环境边界

当前系统使用模拟 TEE，远程证明仅建模输入与输出的绑定关系。不覆盖真实硬件侧信道攻击、远程证明格式兼容性、宿主输入输出操控和供应链信任。因此，本文实验验证的是机制接口能否跑通，而非硬件安全强度。

### 8.2 数据可用性边界

当前使用可验证数据可用性注册表，实现了数据可用性根、负载哈希、字段级 Merkle 证明和可用性状态记录。未接入 Celestia、EigenDA 或其他真实数据可用性网络，因而不证明真实数据可用性经济安全或大规模采样下的可用性保证。

### 8.3 成本模型边界

当前成本模型为配置化估算。全链上调用数据、紧凑模块化数据可用性采样等路线下的燃气估算来自预设参数而非主网交易回执或真实数据可用性网络定价。96.33% 等降幅比例是给定配置下的趋势结果，不模拟主网拥堵费率、blob 基础费率和交易调度延迟。

### 8.4 链上实现边界

Hardhat 本地以太坊虚拟机燃气基线说明 Solidity 原型的提交、挑战、恢复、重放和结算路径具有合约对应物。Sepolia 部署说明核心合约具有公开测试网可部署性。两者均不代表主网性能、审计就绪性或生产可用性。

当前原型尚未测量多并发挑战会话、恢复队列增长、重放队列延迟和调度竞争；并发恢复与重放调度仍属于后续工作。

### 8.5 相关工作

TEEROLLUP 提出了利用异构 TEE 降低汇总验证成本的系统设计。本文受其思路启发，但聚焦异常场景下的挑战恢复和紧凑提交的证据绑定。

opML 等工作研究如何通过 乐观欺诈证明 支持机器学习或大模型计算，通常包含二分定位、单步仲裁和链上虚拟机验证。本文借鉴交互式争议定位思想，但没有实现完整 FPVM 或真实 ML 执行证明，而是将 二分重放用于 Hybrid TEE-Rollup 原型中的异常仲裁。

Dynamic Fraud Proof 关注如何缩短无争议场景下的最终性，并在发现争议时动态延迟结算。该方向强调 挑战窗口 和验证者参与机制的动态调整。本文不提出完整动态最终性协议，而是在更小范围内研究 超时后 挑战会话 如何恢复推进。

惰性账本和模块化数据可用性工作强调数据可用性与执行验证解耦。本文使用可验证数据可用性注册表和配置建模紧凑提交加数据可用性验证的成本趋势。

表 11 相关工作与本文工作的对照

| 方向 / 工作                       | 核心关注                                   | 与本文关系                                               | 本文不声称覆盖的部分                          |
|-------------------------------|----------------------------------------|-----------------------------------------------------|-------------------------------------|
| TEEROLLUP / Hybrid TEE-Rollup | 利用异构 TEE 降低 Rollup 验证成本，并保留挑战和 DA 处罚机制 | 本文沿用 Hybrid TEE-Rollup 的研究方向，聚焦 可恢复挑战路径 与 携带证据的紧凑提交 | 不实现完整 TEE 委员会、生产级提现流程或完整经济安全        |
| 乐观 TEE 汇总 / OTR               | 将 TEE 快速执行与乐观验证 结合                     | 本文同样采用“正常路径低成本、异常路径挑战”的思想                           | 不证明真实 TEE 远程证明机制安全或链上生成式 AI 生产服务可用性 |
| opML / optimistic ML 欺诈证明     | 通过二分定位和单步证明验证 ML 或复杂计算                 | 本文借鉴 二分重放 的争议定位结构                                   | 不实现完整 FPVM、真实模型推理单步证明或链上 ML 执行环境    |
| Dynamic 欺诈证明                  | 动态调整争议窗口和最终性，以缩短无争议路径延迟                | 本文关注 超时 后会话恢复，是更窄的挑战活性 问题                           | 不提出完整动态最终性协议                        |
| LazyLedger / 模块化 DA           | 将 DA 与执行验证解耦，支持数据可用性抽样                 | 本文用 可验证数据可用性 与 数据可用性配置 说明 紧凑提交 需要绑定 DA 证据           | 不接入真实 DA 网络，也不证明 DAS 经济安全           |
| 轻客户端 for Lazy Blockchains     | 面向轻客户端的数据可用性抽样与验证                      | 本文借鉴“轻量验证者需要可检查数据承诺”的思想                             | 不实现真实轻客户端协议                         |
| EIP-4844 / blob 数据            | 降低 Rollup 数据发布成本，提供 blob 数据市场          | 本文将 类 EIP-4844 方案 作为配置化成本 配置                        | 不模拟真实 blob 基础费率、主网拥堵或 blob 生        |

| 方向 / 工作 | 核心关注 | 与本文关系 | 本文不声称覆盖的部分 |
|---------|------|-------|------------|
|         |      |       | 命周期        |

从对照可以看出，已有工作已经提供 TEE-Rollup、乐观欺诈证明 和 模块化数据可用性 等主要路线；本文将 可恢复挑战路径、数据可用性绑定 紧凑提交 和轻量成本模型放入同一个可运行研究原型中，并分别用 Python、Hardhat 与 Sepolia 证据说明机制可行性、链上对应物和可部署性边界。

### 8.6 小结与未来方向

高频去中心化应用的核心困难在于低成本交互与异常可验证之间的持续权衡。本文在已有 Hybrid TEE-Rollup 思路下回答了一个更窄的问题：能否在不重做完整汇总系统的前提下，让轻量提交、数据可用性证据和可恢复挑战形成可运行的研究原型。现有证据给出了谨慎但有意义的回答：Python 原型跑通了紧凑提交、数据可用性证明、挑战、恢复、重放与结算；对比型实验区分了检测与完成、压缩与证据绑定；本地以太坊虚拟机和 Sepolia 证据说明协议路径具有合约对应物。

从理论层面看，本文的主要价值在于将挑战恢复从工程操作提升为具有协议语义的独立机制，提出并论证了仲裁连续性这一协议性质。本文通过协议性质分析（第 4.4 节）系统化了挑战事实不变性、安全性与活性的分离、仲裁连续性和恢复操作正确性四项性质，为后续将该机制推广到更复杂的汇总协议栈提供了理论参照。

后续研究可以沿两个方向推进：一是在真实 TEE 和数据可用性环境中重新测量成本、活性和异常处理边界；二是加强重放证据的链上可验证性，完善故障到罚没策略，补齐并发挑战和恢复频率限制等工程机制。

- [1] WEN X, FENG Q, LYU H, et al. TEEROLLUP: Efficient Rollup Design Using Heterogeneous TEE[EB/OL]. arXiv:2409.14647, 2024.
- [2] PICCO G, FORTUGNO A. Dynamic Fraud Proof[EB/OL]. arXiv:2502.10321, 2025.
- [3] AL-BASSAM M. LazyLedger: A Distributed Data Availability Ledger with Client-Side Smart Contracts[EB/OL]. arXiv:1905.09274, 2019.
- [4] TAS E N, TSE D, YANG L, et al. Light Clients for Lazy Blockchains[EB/OL]. arXiv:2203.15968, 2022.
- [5] Ethereum Foundation. EIP-4844: Shard Blob Transactions[EB/OL]. 2024.
- [6] Ethereum Foundation. Optimistic Rollups[EB/OL]. Ethereum Documentation, 2024.
- [7] Buterin V. An Incomplete Guide to Rollups[EB/OL]. 2021.
- [8] Intel Corporation. Intel Software Guard Extensions (Intel SGX) Developer Guide[EB/OL]. 2024.
- [9] Intel Corporation. Intel Trust Domain Extensions (Intel TDX) Architecture Specification[EB/OL]. 2024.
- [10] Offchain Labs. Arbitrum: A Next-Generation Layer 2 for Ethereum[EB/OL]. 2021.
- [11] Cartesi. Cartesi Rollups Documentation[EB/OL]. 2024.
- [12] EigenLayer. EigenDA Documentation[EB/OL]. 2024.
- [13] Celestia. Celestia: Modular Blockchain Network[EB/OL]. 2024.
- [14] RISC Zero. RISC Zero zkVM Documentation[EB/OL]. 2024.
- [15] Succinct Labs. SP1: A Performant, 100% Open-Source zkVM[EB/OL]. 2024.
- [16] Arbitrum Foundation. Arbitrum Nitro Technical Documentation[EB/OL]. 2024.
- [17] OP Labs. Optimism Documentation: Fault Proofs and Dispute Games[EB/OL]. 2024.
- [18] Canetti R, et al. Universally Composable Security: A New Paradigm for Cryptographic Protocols[EB/OL]. 2000.
- [19] Goldwasser S, Micali S, Rackoff R. The Knowledge Complexity of Interactive Proof Systems[J]. SIAM Journal on Computing, 1989, 18(1): 186-208.
- [20] NIST. Recommendation for Key Management: Part 1 - General (Rev. 5)[EB/OL]. 2020.